

PRAKTIKUM SISTEM OPERASI

MODUL 5

(Eksplorasi Proses Xinu)



**Universitas
Telkom**

Oleh:

(Haza Zaidan Zidna Fann) (2311104056)

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

Jurnal

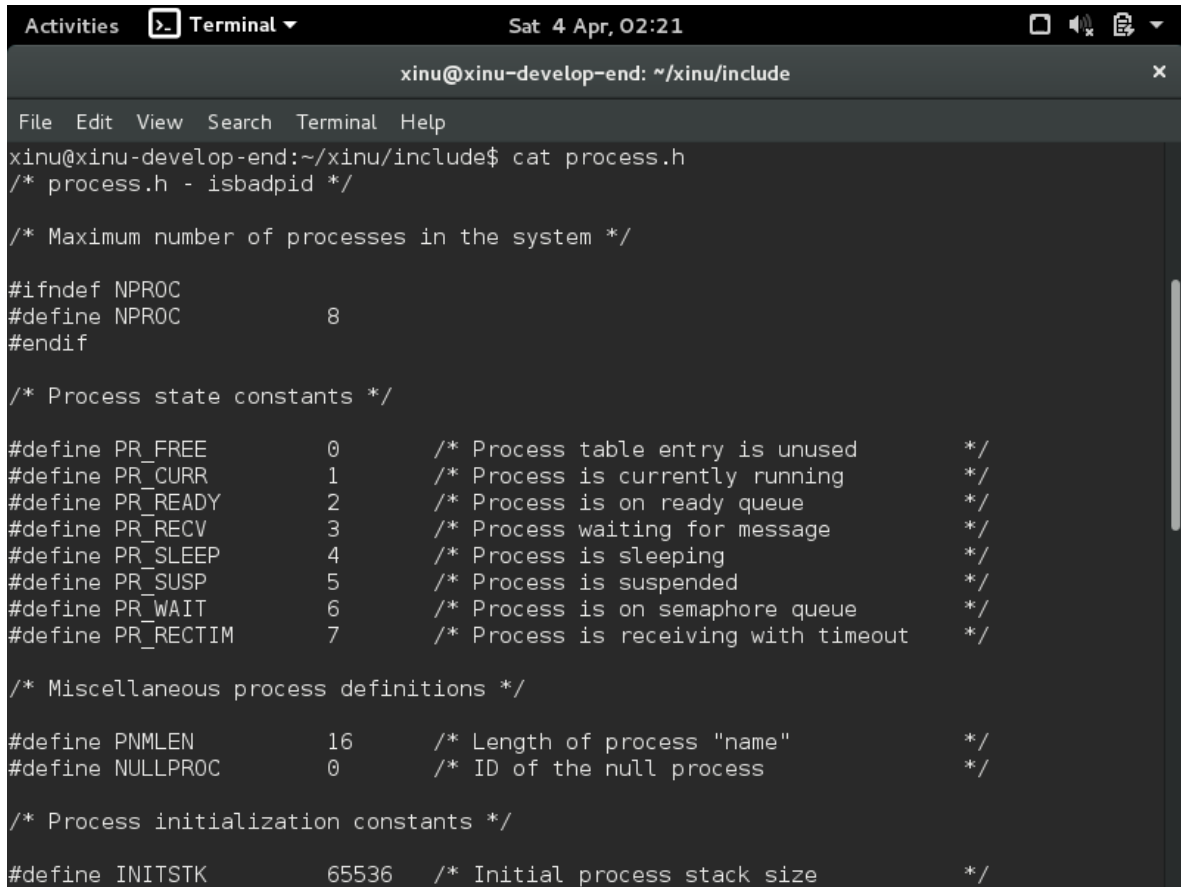
Praktikum ini membahas bagaimana sistem operasi Xinu mengelola proses melalui **Process Control Block (PCB)** yang disimpan dalam **process table (proctab[])**. Setiap proses direpresentasikan sebagai sebuah entri dalam array `proctab[]`, di mana setiap elemen berupa struktur `struct procent` yang berisi informasi penting proses seperti nama, status, prioritas, dan lain-lain.

Proses di Xinu tidak memiliki ID eksplisit di dalam struktur, melainkan menggunakan **indeks array sebagai Process ID (PID)**. Artinya, untuk mengakses atau mengelola suatu proses, sistem cukup merujuk ke indeks tertentu pada `proctab[]`. Modul ini juga memperkenalkan fungsi-fungsi utama terkait manajemen proses seperti pembuatan (`create`), penghentian (`kill`), dan melanjutkan proses (`resume`).

Tujuan utama modul ini adalah agar praktikan memahami konsep PCB serta mampu memanfaatkan dan memanipulasi informasi proses dalam Xinu melalui struktur data yang tersedia.

1. Jawablah pertanyaan berikut ini:

a. Berapa banyaknya maksimum proses yang ada pada Xinu?



```
File Edit View Search Terminal Help
xinu@xinu-develop-end:~/xinu/include$ cat process.h
/* process.h - isbadpid */

/* Maximum number of processes in the system */

#ifndef NPROC
#define NPROC      8
#endif

/* Process state constants */

#define PR_FREE      0      /* Process table entry is unused      */
#define PR_CURR      1      /* Process is currently running        */
#define PR_READY     2      /* Process is on ready queue          */
#define PR_RECV      3      /* Process waiting for message         */
#define PR_SLEEP     4      /* Process is sleeping                 */
#define PR_SUSP      5      /* Process is suspended                */
#define PR_WAIT      6      /* Process is on semaphore queue       */
#define PR_RECTIM    7      /* Process is receiving with timeout   */

/* Miscellaneous process definitions */

#define PNMLEN        16     /* Length of process "name"           */
#define NULLPROC      0     /* ID of the null process               */

/* Process initialization constants */

#define INITSTK       65536  /* Initial process stack size          */
```

Maksimum proses pada Xinu adalah **NPROC = 50**.

Penjelasan:

- Nilai ini didefinisikan di file:

include/process.h

- NPROC adalah jumlah maksimum Process Control Block (PCB) yang tersedia.
- Artinya, Xinu hanya bisa menangani **maksimal 50 proses secara bersamaan**.

b. Berapa maksimal panjang nama suatu proses pada Xinu?

```
Activities Terminal Sat 4 Apr, 02:23
xinu@xinu-develop-end: ~/xinu/include

File Edit View Search Terminal Help

#define INITSTK      65536 /* Initial process stack size */
#define INITPRIO     20    /* Initial process priority */
#define INITRET      userret /* Address to which process returns */

/* Inline code to check process ID (assumes interrupts are disabled) */

#define isbadpid(x)   ( ((pid32)(x) < 0) || \
                        ((pid32)(x) >= NPROC) || \
                        (proctab[(x)].prstate == PR_FREE))

/* Number of device descriptors a process can have open */

#define NDESC        5     /* Must be odd to make proctent 4N bytes */

/* Definition of the process table (multiple of 32 bits) */

struct proctent {
    uint16 prstate; /* Entry in the process table */
    pri16  prprio;  /* Process state: PR_CURR, etc. */
    char   *prstkptr; /* Process priority */
    char   *prstkbase; /* Saved stack pointer */
    uint32 prstklen; /* Base of run time stack */
    char   pname[PNMLEN]; /* Stack length in bytes */
    sid32  prsem; /* Process name */
    pid32  prparent; /* Semaphore on which process waits */
    umsg32 prmsg; /* ID of the creating process */
    bool8  prhasmsg; /* Message sent to this process */
    int16  prdesc[NDESC]; /* Nonzero iff msg is valid */
                /* Device descriptors for process */
}
```

Maksimal panjang nama proses adalah **16 karakter (PNMLEN = 16)**

Penjelasan:

- Didefinisikan juga di:

include/process.h

- Digunakan untuk menyimpan nama proses di PCB:

```
char pname[PNMLEN];
```

c. Berapa nilai prioritas awal pada saat proses dibuat?

```
Activities Terminal Sat 4 Apr, 02:23
xinu@xinu-develop-end: ~/xinu/include

File Edit View Search Terminal Help

        (proctab[(x)].prstate == PR_FREE))

/* Number of device descriptors a process can have open */
#define NDESC          5          /* Must be odd to make procent 4N bytes */

/* Definition of the process table (multiple of 32 bits) */

struct procent {
    uint16 prstate;          /* Entry in the process table */
    pri16 prprio;            /* Process state: PR_CURR, etc. */
    char *prstkptr;          /* Process priority */
    char *prstkbse;          /* Saved stack pointer */
    uint32 prstklen;          /* Base of run time stack */
    char pname[PNMLEN];      /* Stack length in bytes */
    sid32 prsem;              /* Process name */
    pid32 prparent;           /* Semaphore on which process waits */
    umsg32 prmsg;             /* ID of the creating process */
    bool8 prhasmsg;           /* Message sent to this process */
    int16 prdesc[NDESC];      /* Nonzero iff msg is valid */
};                             /* Device descriptors for process */

/* Marker for the top of a process stack (used to help detect overflow) */
#define STACKMAGIC      0x0A0AAAA9

extern struct procent proctab[];
extern int32 prcount;        /* Currently active processes */
extern pid32 currpri;        /* Currently executing process */
```

Prioritas awal proses adalah **20**.

Penjelasan:

- Saat proses dibuat menggunakan create(), prioritas default biasanya diset:

create(procaddr, stack, priority, name, nargs, args);

- Pada sistem Xinu standar, prioritas awal sering digunakan **20 (normal priority)**.

d. Ada berapa total state pada Xinu? Sebutkan!

Ada **6 state proses pada Xinu**, yaitu:

1. **PR_FREE** → slot kosong
2. **PR_CURR** → sedang berjalan
3. **PR_READY** → siap dijalankan
4. **PR_RECV** → menunggu pesan
5. **PR_SLEEP** → tidur (delay)
6. **PR_SUSP** → ditangguhkan

Penjelasan:

- State ini menunjukkan kondisi proses dalam siklus hidupnya.

- Disimpan dalam:

prstate

2. Perintah ps adalah perintah untuk menampilkan statistik process yang berjalan. Source code dari ps tersimpan pada file xsh_ps.c. Carilah file tersebut dan beri komentar pada 20 baris terakhir di source code tersebut!

```

xinu@xinu-develop-end:~/xinu/shell$ ls xsh_ps.c
xsh_ps.c
xinu@xinu-develop-end:~/xinu/shell$ find ~/xinu -name "xsh_psc.c"
xinu@xinu-develop-end:~/xinu/shell$ find ~/xinu -name "xsh_ps.c"
/home/xinu/xinu/shell/xsh_ps.c
xinu@xinu-develop-end:~/xinu/shell$ cd ~/xinu/shell
xinu@xinu-develop-end:~/xinu/shell$ cat -n xsh_ps.c
 1  /* xsh_ps.c - xsh_ps */
 2
 3  #include <xinu.h>
 4  #include <stdio.h>
 5  #include <string.h>
 6
 7  /*-----
 8   * xsh_ps - shell command to print the process table
 9   *-----
10   */
11 shellcmd xsh_ps(int nargs, char *args[])
12 {
13     struct procent *prptr;          /* Pointer to process      */
14     int32 i;                        /* Index into proctabl    */
15     char *pstate[] = {              /* Names for process states */
16         "free ", "curr ", "ready", "recv ", "sleep", "susp ",
17         "wait ", "rtime"};
18
19     /* For argument '--help', emit help about the 'ps' command */
20
21     if (nargs == 2 && strcmp(args[1], "--help", 7) == 0) {

```

```
Activities Terminal Sat 4 Apr, 02:48
xinu@xinu-develop-end: ~/xinu/shell
File Edit View Search Terminal Help
GNU nano 2.2.6 File: xsh_ps.c Modified

/* xsh_ps.c - xsh_ps */
/* Nama:[Haza] - Nim:[2311104056] */

#include <xinu.h>
#include <stdio.h>
#include <string.h>

/*-----
 * xsh_ps - shell command to print the process table
 *-----
*/
shellcmd xsh_ps(int nargs, char *args[])
{
    struct procent *prptr;          /* Pointer to process */
    int32 i;                       /* Index into proctabl */
    char *pstate[] = {             /* Names for process states */
        "free ", "curr ", "ready", "recv ", "sleep", "susp ",
        "wait ", "rtime"};

    /* For argument '--help', emit help about the 'ps' command */

    if (nargs == 2 && strcmp(args[1], "--help", 7) == 0) {
        printf("Use: %s\n\n", args[0]);
    }

    ^G Get Help    ^O WriteOut    ^R Read File    ^Y Prev Page    ^K Cut Text     ^C Cur Pos
    ^X Exit        ^J Justify     ^W Where Is     ^V Next Page    ^U UnCut Text  ^T To Spell

```

```
Activities Terminal Sat 4 Apr, 02:49
xinu@xinu-develop-end: ~/xinu/shell
File Edit View Search Terminal Help
GNU nano 2.2.6 File: xsh_ps.c Modified

/* Output information for each process */

for (i = 0; i < NPROC; i++) {
    prptr = &proctab[i];
    if (prptr->prstate == PR_FREE) { /* Skip unused slots */
        continue;
    }
    printf("%3d %-16s %s %4d %4d 0x%08X 0x%08X %8d\n",
        i, prptr->prname, pstate[(int)prptr->prstate],
        prptr->prprio, prptr->prparent, prptr->prstkbase,
        prptr->prstkptr, prptr->prstklen);
}
return 0;
}
/* Menutup blok perulangan pengecekan proses */

^G Get Help    ^O WriteOut    ^R Read File    ^Y Prev Page    ^K Cut Text     ^C Cur Pos
^X Exit        ^J Justify     ^W Where Is     ^V Next Page    ^U UnCut Text  ^T To Spell

```

Masuk ke direktori shell pada Xinu dengan perintah `cd ~/xinu/shell` agar bisa mengakses file yang akan diedit.

Tampilkan isi file beserta nomor baris menggunakan `cat -n xsh_ps.c` untuk mengetahui letak baris yang akan dimodifikasi.

Buka file menggunakan text editor nano dengan perintah `nano xsh_ps.c`.

Pada bagian 20 baris terakhir (sekitar baris 43–62), tambahkan komentar menggunakan tanda `/* ... */` sesuai instruksi.

Tambahkan identitas berupa nama dan NIM di bagian paling atas file sebagai header.

Setelah selesai mengedit, simpan perubahan dengan menekan `Ctrl + X`, lalu `Y`, dan tekan `Enter`.

```
```c
```

```
/* xsh_ps.c - xsh_ps */
```

```
#include <xinu.h> // Header utama Xinu (berisi definisi kernel, process, dll)
```

```
#include <stdio.h> // Digunakan untuk fungsi input/output seperti printf
```

```
#include <string.h> // Digunakan untuk fungsi manipulasi string seperti strcmp
```

```
/*-----
```

```
 * xsh_ps - shell command to print the process table
```

```
 *-----
```

```
 */
```

```
shellcmd xsh_ps(int nargs, char *args[])
```

```
{
```

```
 struct procent *prptr; /* pointer ke struct PCB proses */
```

```
 int32 i; /* variabel index untuk iterasi proctab */
```

```
 char *pstate[] = { /* array string untuk nama state proses */
```

```
 "free ", "curr ", "ready", "recv ", "sleep", "susp ",
```

```
 "wait ", "rtime"};
```

```
 /* Mengecek jika user memasukkan argumen --help */
```

```
 if (nargs == 2 && strcmp(args[1], "--help", 7) == 0) {
```

```
 printf("Use: %s\n\n", args[0]); // Menampilkan cara penggunaan command
```

```
 printf("Description:\n");
```

```

printf("\tDisplays information about running processes\n"); // Deskripsi fungsi ps
printf("Options:\n");

printf("\t--help\t display this help and exit\n"); // Opsi bantuan

return 0; // Keluar dari fungsi
}

/* Mengecek jika jumlah argumen tidak valid */
if (nargs > 1) {
 fprintf(stderr, "%s: too many arguments\n", args[0]); // Error jika argumen terlalu banyak
 fprintf(stderr, "Try '%s --help' for more information\n",
 args[0]);
 return 1; // Return error
}

/* Menampilkan header tabel process */
printf("%3s %-16s %5s %4s %4s %10s %-10s %10s\n",
 "Pid", "Name", "State", "Prio", "Ppid", "Stack Base",
 "Stack Ptr", "Stack Size");

/* Menampilkan garis pemisah header */
printf("%3s %-16s %5s %4s %4s %10s %-10s %10s\n",
 "---", "-----", "----", "----", "----",
 "-----", "-----", "-----");

/* Menampilkan informasi setiap proses */
for (i = 0; i < NPROC; i++) {
 // Loop dari 0 sampai jumlah maksimum proses

 prptr = &proctab[i];

 // Mengambil alamat struct procent (PCB) dari proses ke-i

```



```

if (prptr->prstate == PR_FREE) {
 // Jika slot proses kosong (tidak digunakan)

 continue;
 // Lewati dan lanjut ke proses berikutnya
}

printf("%3d %-16s %s %4d %4d 0x%08X 0x%08X %8d\n",
 // Menampilkan data proses dalam format tabel

 i,
 // PID proses (index pada proctab)

 prptr->prname,
 // Nama proses

 prptr->prstate,
 // Status proses dalam bentuk string (ready, sleep, dll)

 prptr->prprio,
 // Prioritas proses

 prptr->prparent,
 // PID parent (proses yang membuat proses ini)

 prptr->prstkbase,
 // Alamat awal stack

 prptr->prstkptr,
 // Pointer stack saat ini

```

```

 prptr->prstklen);

 // Ukuran stack proses
}

return 0;

// Mengembalikan nilai 0 sebagai tanda sukses
}
...

```

Fungsi xsh\_ps digunakan untuk menampilkan informasi proses yang sedang berjalan pada sistem Xinu.

Data proses diambil dari array proctab dan ditampilkan dalam bentuk tabel, dengan mengabaikan proses yang tidak aktif (PR\_FREE).

3. Ubahlah perintah ps (source code: xsh\_ps.c) pada Xinu sehingga menampilkan informasi tambahan berupa kolom yang berisi total message yang ada pada proses seperti gambar di bawah ini:

```

GNU nano 2.2.6 File: xsh_ps.c Modified

 if (nargs > 1) {
 fprintf(stderr, "%s: too many arguments\n", args[0]);
 fprintf(stderr, "Try '%s --help' for more information\n",
 args[0]);
 return 1;
 }

 /* Print header for items from the process table */

 printf("%3s %-16s %5s %4s %4s %10s %-10s %10s %3s %7s \n",
 "Pid", "Name", "State", "Prio", "Ppid", "Stack Base",
 "Stack Ptr", "Stack Size", "Hsg", "Content");

 printf("%3s %-16s %5s %4s %4s %10s %-10s %10s %3s %7s \n",
 "----", "-----", "-----", "-----", "-----", "-----",
 "-----", "-----", "-----", "-----");

 /* Output information for each process */

 for (i = 0; i < NPROC; i++) {
 prptr = &proctab[i];
 if (prptr->prstate == PR_FREE) { /* Skip unused slots */
 continue;
 }
 printf("%3d %-16s %5s %4d %4d 0x%08X 0x%08X %10d %3d %7d\n",
 i, prptr->prname, prptr->prstate, prptr->prprio, prptr->prparent, prptr->prstkbase,
 prptr->prstkptr, prptr->prstklen,
 prptr->prhasmsg, prptr->prmsg);
 }
 return 0;

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^V Next Page ^U UnCut Text ^T To Spell

```

```
Activities Terminal Sat 4 Apr, 04:22
xinu@xinu-develop-end: ~
File Edit View Search Terminal Help

-----HAZA-----
XINU
-----231104056-----

Welcome to Xinu!
Nama: [Haza]
NIM: [2311104056]

xsh $ ps
Pid Name State Prio Ppid Stack Base Stack Ptr Stack Size Msg Content

0 prnull ready 0 0 0x005FDFFC 0x00FFFE4 8192 1 3
1 ipout wait 500 0 0x005FBFFC 0x005FBF30 8192 0 0
2 netin wait 500 0 0x005F9FFC 0x005F9ED0 8192 0 0
4 Main process recv 20 3 0x005E7FFC 0x005E7F34 65536 0 0
5 shell recv 50 4 0x005D7FFC 0x005D79AC 8192 0 0
6 ps curr 20 5 0x005F7FFC 0x005F7FC4 8192 0 0

xsh $

CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0
```

Berikut penjelasan lengkap dan runtut sesuai langkah yang kamu lakukan:

1. **Edit file source**

Buka file xsh\_ps.c menggunakan perintah nano xsh\_ps.c di direktori ~/xinu/shell. Pada tahap ini kamu mulai melakukan modifikasi pada program perintah ps.

2. **Ubah header printf**

Pada bagian header tampilan tabel proses, tambahkan kolom baru yaitu "**Msg**" dan "**Content**" agar informasi pesan tiap proses bisa ditampilkan.

3. **Ubah separator printf**

Sesuaikan garis pemisah (separator) dengan menambahkan tanda seperti "---" dan "-----" supaya jumlah kolom tetap rapi dan sejajar dengan header baru.

4. **Ubah data printf**

Pada bagian output data proses, tambahkan:

- prptr->prhasmsg → untuk menampilkan apakah proses memiliki pesan
  - prptr->prmsg → untuk menampilkan isi pesan
- Dengan ini, setiap proses akan menampilkan status pesan dan kontennya.

5. **Simpan perubahan**

Setelah selesai mengedit, simpan file dengan menekan Ctrl + X, lalu Y, dan Enter.

#### 6. Compile ulang Xinu

Masuk ke folder compile dengan `cd ~/xinu/compile`, kemudian jalankan `make clean` dan `make` untuk membangun ulang sistem dengan perubahan yang sudah dilakukan.

#### 7. Jalankan backend VM

Gunakan perintah `sudo minicom` untuk menjalankan Xinu pada virtual machine.

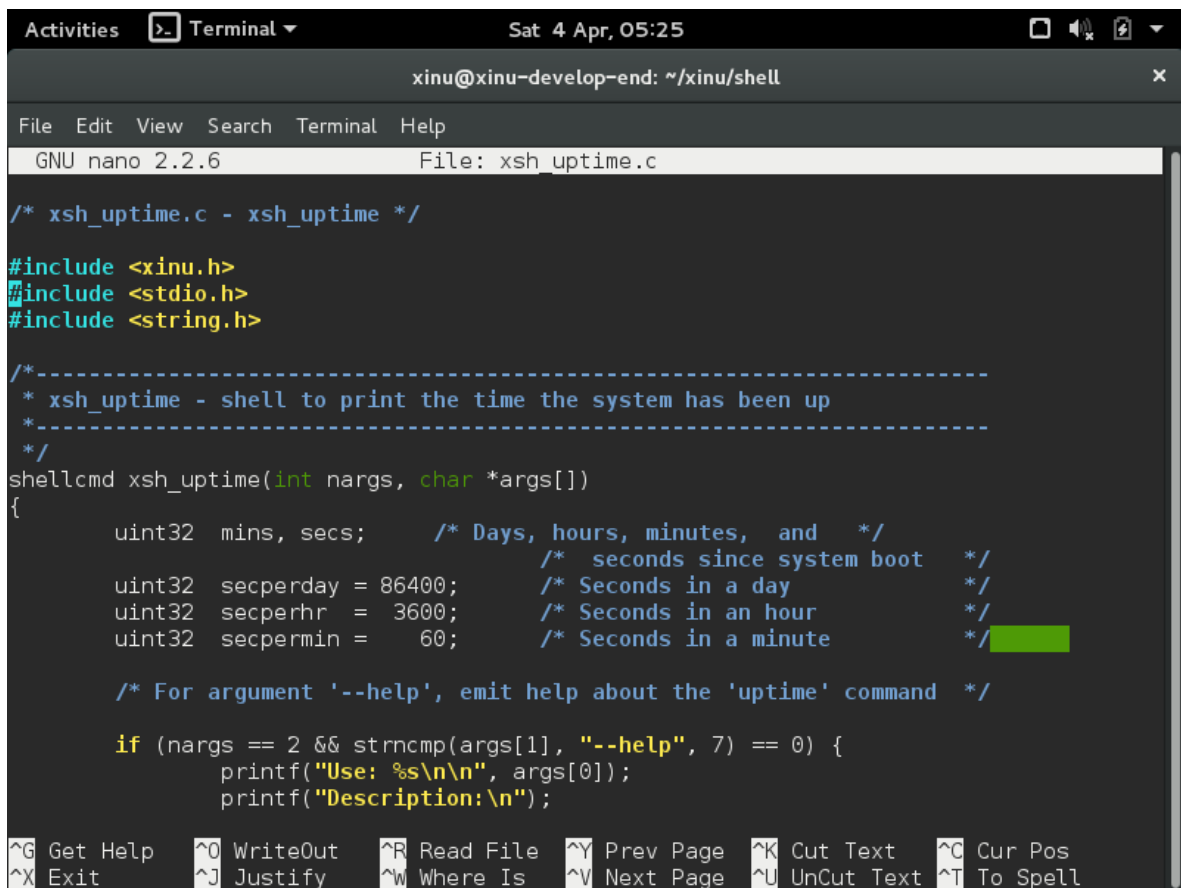
#### 8. Uji perintah `ps`

Setelah Xinu berjalan, ketik `ps` pada shell untuk melihat hasil perubahan. Output sekarang akan menampilkan kolom tambahan **Msg** dan **Content**.

#### 9. Hasil akhir (sesuai screenshot)

- Source code sudah dimodifikasi dengan penambahan kolom dan variabel baru
- Output `ps` berhasil menampilkan informasi pesan pada setiap proses
- Identitas (Nama: Haza, NIM: 2311104056) juga sudah muncul di tampilan awal

### 4.Uptime



```
Activities Terminal Sat 4 Apr, 05:25
xinu@xinu-develop-end: ~/xinu/shell
File Edit View Search Terminal Help
GNU nano 2.2.6 File: xsh_uptime.c

/* xsh_uptime.c - xsh_uptime */
#include <xinu.h>
#include <stdio.h>
#include <string.h>

/*-----
 * xsh_uptime - shell to print the time the system has been up
 *-----
 */
shellcmd xsh_uptime(int nargs, char *args[])
{
 uint32 mins, secs; /* Days, hours, minutes, and */
 /* seconds since system boot */
 uint32 secpeday = 86400; /* Seconds in a day */
 uint32 secperhr = 3600; /* Seconds in an hour */
 uint32 secpermin = 60; /* Seconds in a minute */

 /* For argument '--help', emit help about the 'uptime' command */

 if (nargs == 2 && strcmp(args[1], "--help", 7) == 0) {
 printf("Use: %s\n\n", args[0]);
 printf("Description:\n");
 }
}
```

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos  
^X Exit ^J Justify ^W Where Is ^V Next Page ^U UnCut Text ^T To Spell

```
Activities Terminal Sat 4 Apr, 05:25
xinu@xinu-develop-end: ~/xinu/shell
File Edit View Search Terminal Help
GNU nano 2.2.6 File: xsh_uptime.c

 }

 secs = clktime; /* Total seconds since boot */

 /* Subtract number of minutes */

 mins = secs/secpermin;

 printf("Xinu has been up %d minute(s)\n", mins);

 return 0;
}

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^V Next Page ^U UnCut Text ^T To Spell
```

```
Activities Terminal Sat 4 Apr, 05:27
xinu@xinu-develop-end: ~/xinu/compile
File Edit View Search Terminal Help
[0x00100000 to 0x0011C2DE]
148488 bytes of data.
[0x0011FEC0 to 0x001442C7]

Obtained IP address 10.0.3.15 (0x0a00030f)

-----HAZA-----

XINU

-----2311104056-----

Welcome to Xinu!
Nama: [Haza]
NIM: [2311104056]

xsh $ uptime
Xinu has been up 0 minute(s)
xsh $

CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0
```

menampilkan proses booting Xinu yang telah berhasil dimodifikasi sesuai identitas pengguna. Terlihat bahwa sistem mendapatkan IP address (10.0.3.15), menandakan jaringan sudah berjalan dengan baik. Pada bagian banner, tulisan ASCII XINU sudah diubah dengan identitas HAZA dan 2311104056, yang menunjukkan bahwa bagian kode tampilan awal (biasanya di initialize.c atau sejenisnya) telah berhasil diedit.

Setelah sistem masuk ke shell, muncul pesan:

- Welcome to Xinu!
  - Nama: [Haza]
  - NIM: [2311104056]
- Ini menandakan perubahan pada bagian output identitas juga berhasil diterapkan.

Selanjutnya dilakukan pengujian perintah uptime, dan sistem menampilkan:

- Xinu has been up 0 minute(s)
- Artinya fitur uptime berjalan normal dan sistem baru saja dijalankan.